



Chapitre 1 :

Gestion et Ordonnancement des processus



1

1



Plan

- **Ordonnancement des processus**



2

2

Ordonnancement des processus

3

3

Le Cycle de Vie d'un Processus

Les 5 États Clés:

1. **Nouveau (New):** Le processus est en cours de création.
2. **Prêt (Ready):** Le processus attend son tour dans la file pour accéder au processeur.
3. **En exécution (Running):** Le processeur est en train d'exécuter les instructions du processus.
4. **En attente (Waiting/Blocked):** Le processus est «bloqué» car il attend un événement, comme une entrée utilisateur ou la lecture d'un fichier.
5. **Terminé (Terminated):** Le processus a fini son travail et libère les ressources.

4

4

Ordonnancement des processus

➤ Prêt → Exécution

- Lorsque l'ordonnanceur UCT (CPU scheduler) choisit un processus pour exécution

➤ Exécution → Prêt

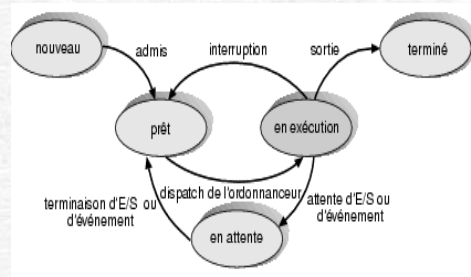
- Résultat d'une interruption causée par un événement indépendant du processus

➤ Exécution → Attente

- Lorsqu'un processus fait requête d'un service du SE que le SE ne peut offrir immédiatement (interruption causée par le processus lui-même)
 - un accès à une ressource pas encore disponible
 - initie une E/S: doit attendre le résultat
 - a besoin de la réponse d'un autre processus

➤ Attente → Prêt

- lorsque l'événement attendu se produit



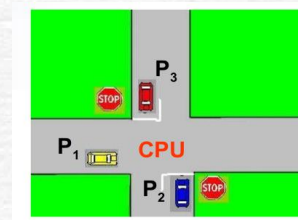
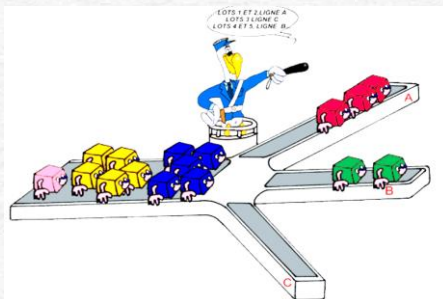
5

5

Ordonnancement des processus

▪ Plusieurs processus sont prêts à être exécutés

- Comment contrôler l'ordre de passage des processus sur le processeur ?
- Comment contrôler la répartition du temps d'exécution entre les processus ?



Ordonnanceur

- L'ordonnanceur est un composant du système d'exploitation

6

6

Ordonnancement des processus

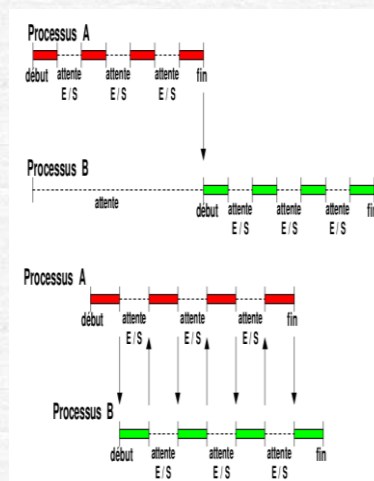
- Le SE doit faire un choix (Pq ordonnancer ?) :
 - **Équité** : ne pas laisser un processus attendre indéfiniment
 - **Efficacité** : le processeur doit être utilisé à 100%
 - **Temps de réponse** : l'utilisateur devant sa machine ne doit pas trop attendre
 - **Temps d'exécution** : une séquence d'instructions ne doit pas trop durer
 - **Rendement** : il faut faire le plus de choses en une heure
- Ordonnancement sans réquisition: un processus est exécuté jusqu'à la fin
 - Inefficace et dangereux (ex: exécution d'une boucle sans fin...)
- Ordonnancement avec réquisition: le système d'exploitation peut interrompre de force un processus en cours d'exécution pour donner le processeur à un autre
 - A chaque signal d'horloge, le SE reprend la main, décide si le processus courant a consommé son quota de temps machine et alloue éventuellement le processeur à un autre processus
 - Il existe de nombreux algorithmes d'ordonnancement avec réquisition

7

7

Ordonnancement des processus

- Longueur normale des cycles
 - Étant donné la grande vitesse de l'UCT par rapport aux périphériques, la plupart des cycles d'UCT seront très courts
 - Cependant il pourrait y avoir des longs cycles d'UCT quand on demandera des calculs poussés, ex. calculs scientifiques
- L'ordonnanceur UCT doit prendre sa décision chaque fois que le processus exécutant est interrompu
 - un processus se présente en tant que nouveau ou se termine
 - un processus exécutant devient bloqué en attente
 - un processus change d'exécutant/running à prêt/ready
 - un processus change de attente à prêt/ready



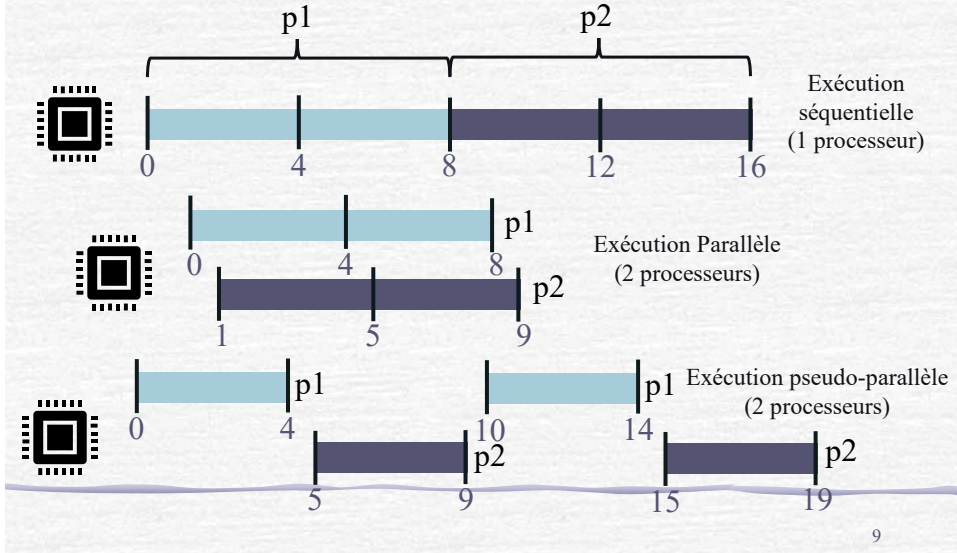
8

8

Ordonnancement des processus

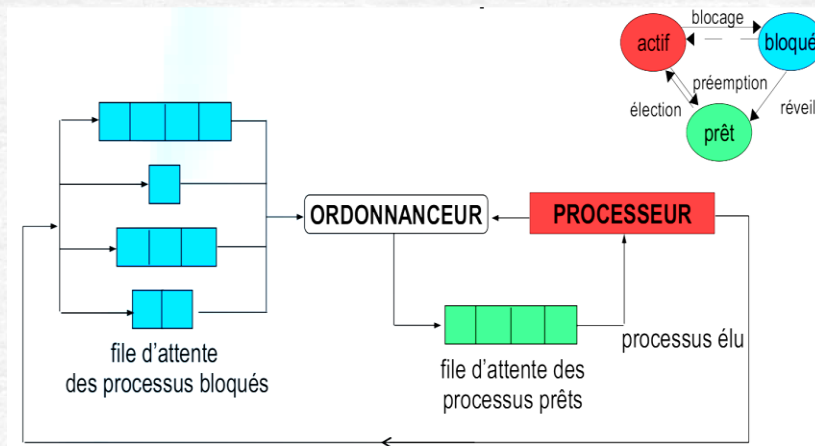
- Soit deux processus p1 et p2 (exécution de deux programmes séquentiels P1 et P2)

- Mise en œuvre concrète de l'exécution de p1 et p2



9

Ordonnancement des processus



ORDONNANCEUR : alloue le processeur aux différents processus selon un algorithme d'ordonnancement donné

10

10

Ordonnancement des processus

- Rôle de l'ordonnanceur => Sa fonction principale est de choisir un processus parmi ceux qui sont "prêts" en mémoire pour lui allouer l'unité centrale (CPU).
- Critères d'Ordonnancement
 - Utilisation de la CPU – utiliser la CPU le maximum possible
 - Débit (Throughput) – # de processus qui terminent leur exécution par unité de temps
 - Temps de rotation (Turnaround time) – le temps depuis le lancement du processus jusqu'à sa terminaison (les attentes incluses)
 - Temps d'attente – temps d'un processus dans la file d'attente des processus prêts
- Critères d'Optimisation
 - Utilisation maximale du CPU
 - Débit maximum
 - Temps de rotation minimal
 - Temps d'attente minimal

11

11

Algorithmes d'ordonnancement

- Ordonnancement non préemptif (sans réquisition)
 - First Come First Serve (FCFS): premier arrivé, premier servi
 - Ordonnancement selon la durée de calcul : le plus court d'abord ou Shortest Job First (SJF)
- Ordonnancement préemptif (avec réquisition)
 - Ordonnancement selon la durée de calcul restante : temps restant le plus court d'abord ou Shortest Remaining Time (SRT)
 - Ordonnancement sans notion de priorité : temps-partagé avec politique du tourniquet ou Round-Robin (RR)
 - Ordonnancement à priorités (statiques ou dynamiques): la tâche la plus prioritaire obtient le processeur

12

12

□ RQ:

- La préemption est le cas normal pour l'UCT car l'arrivée d'un nouveau proc cause une interruption et à ce moment là l'ordonnanceur devra prendre une décision sur le prochain proc à exécuter
- Mais dans quelques ressources la préemption est impossible: p.ex. dans une imprimante il faut toujours terminer l'impression courante avant d'en amorcer une nouvelle

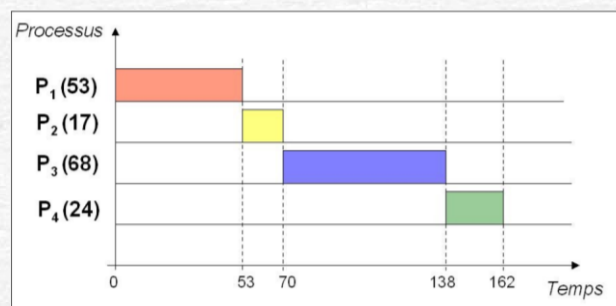
13

13

Premier arrivé, premier servi (FIFO)

➤ Premier arrivé, premier servi (FIFO)

- **Principe:** un processus n'est traité qu'au moment où le précédent se termine. Les processus sont traités dans leur ordre d'arrivée
- **Exemple:** P1, P2, P3 et P4 arrivent dans cet ordre à $t = 0$



14

14

Premier arrivé, premier servi (FIFO)

Processus n°	Temps d'exécution
1	30
2	3
3	6
4	4

- Diagramme de Gantt :

30	33	39	43
P1	P2	P3	P4

- Temps d'attente moyen: $(0 + 30 + 33 + 39)/4 = 25.5$
- Utilisation UCT = 100%
- Débit = $4/43 = 0,1$; 4 processus complétés en 43 unités de temps
- Temps de rotation moyen: $(30 + 33 + 39 + 43)/4 = 36.25$

15

15

Premier arrivé, premier servi (FIFO)

Exercice 1:

- Donner le diagramme de Gantt, le temps de rotation moyen, et le temps moyen d'attente correspondant au scénario suivant. Supposons que les processus arrivent dans l'ordre suivant: P1 , P2 , P3:

Processus	Tps CPU
P_1	24
P_2	3
P_3	3

16

16

Premier arrivé, premier servi (FIFO)

■ Exercice 1:

Processus	Tps CPU
P_1	24
P_2	3
P_3	3

■ Exercice 1: correction

○ Diagramme de Gantt



- Temps d'attente moyen: $(0 + 24 + 27)/3 = 17$
- Utilisation UCT = 100%
- Débit = $3/30 = 0,1$, 3 processus complétés en 30 unités de temps
- Temps de rotation moyen: $(24+27+30)/3 = 27$

17

17

Premier arrivé, premier servi (FIFO)

■ Exercice 2:

- Donner le diagramme de Gantt et le temps moyen d'attente correspondant au scénario suivant, dans le cas où les processus arrivent à 0 mais dans l'ordre P_2, P_3, P_1

Processus	Tps CPU
P_1	24
P_2	3
P_3	3

18

18

Premier arrivé, premier servi (FIFO)

- Exercice 2:
 - Donner le diagramme de Gantt et le temps moyen d'attente correspondant au scénario suivant, dans le cas où les processus arrivent à 0 mais dans l'ordre P_2, P_3, P_1
- Exercice 2: correction



- Temps d'attente pour P₂ = 0 P₃ = 3 P₁ = 6
- Temps moyen d'attente: $(6 + 0 + 3)/3 = 3$ était 17
- Temps de rotation moyen: $(3+6+30)/3 = 13$ était 27
- Beaucoup mieux!
- Donc pour cette technique, les temps peuvent varier grandement par rapport à l'ordre d'arrivée de différents processus

19

19

Premier arrivé, premier servi (FIFO)

- Exercice 3:
 - Donner le diagramme de Gantt, le temps de rotation moyen, et le temps moyen d'attente correspondant au scénario suivant; le cas où les processus arrivent à moment différents:
 - P₁ arrive à temps 0 et dure 24
 - P₂ arrive à temps 2 et dure 3
 - P₃ arrive à temps 5 et dure 3

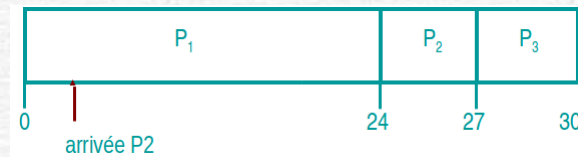
20

20

Premier arrivé, premier servi (FIFO)

■ Exercice 3: correction

- Diagramme de Gantt: P1 attend 0 comme avant, mais P2 attend 24-2, etc.



- Temps d'attente moyen: $(0+(24-2)+(27-5))/3=14.7$
- Utilisation UCT = 100%
- Débit : 3/30, processus complétés en 30 unités de temps
- Temps de rotation moyen: $(24+(27-2)+(30-5))/3=24.6$

21

21

Premier arrivé, premier servi (FIFO)

■ Avantages

- Algorithme facile à comprendre
- Faible complexité d'implémentation

■ Inconvénients

- Pas de prise en compte de l'importance relative des processus
- Temps d'attente généralement important

22

22

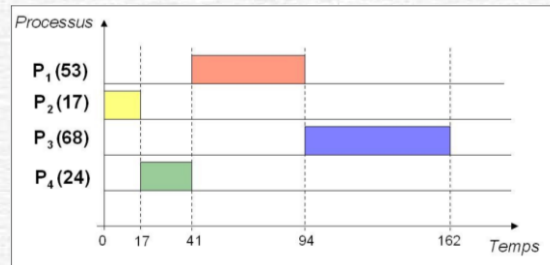
Shortest Job First (SJF)

Plus court temps d'exécution d'abord ou SJF: Shortest Job First

○ Principe:

- associer à chaque processus son prochain temps d'utilisation du CPU
- utiliser ces temps pour choisir le processus avec le temps le plus petit
- le processus dont le temps d'exécution est le plus court est ordonnancé en priorité

○ Exemple: (P1, P2, P3 et P4 arrivent à $t = 0$) :



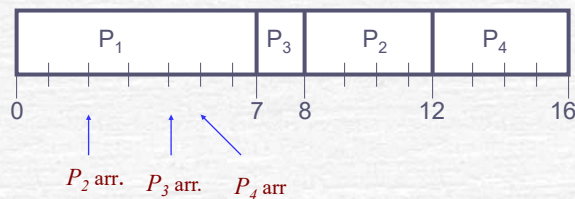
23

23

Shortest Job First (SJF)

○ Exemple

Processus	Tps d'Arrivée	Tps CPU
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4



○ Temps moyen d'attente = $(0 + (8-2) + (7-4) + (12-5)) / 4 = (0 + 6 + 3 + 7) / 4 = 4$

○ Temps de rotation moyen = $(7 + (12-2) + (8-4) + (16-5)) / 4 = 8$

24

24

Shortest Job First (SJF)

➤ Intérêts

- SJF réduit le temps d'attente des processus

➤ Inconvénients

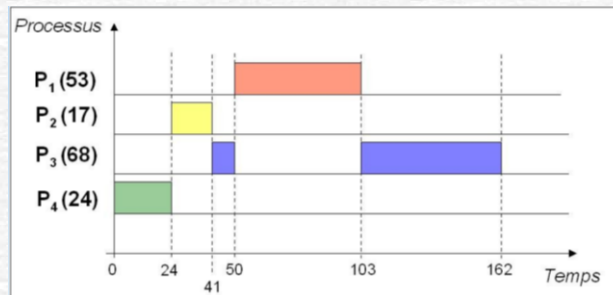
- Pas de prise en compte de l'importance relative des processus
- Algorithme optimal uniquement dans le cas où tous les processus sont disponibles simultanément

25

25

Shortest-Remaining-Time-First (SRTF)

- Plus court temps d'exécution d'abord (PCTE)
 - **Principe:** Le processus dont le temps d'exécution restant est le plus court parmi ceux qui restent à exécuter, est ordonnancé en premier
 - **Exemple:** P3 et P4 arrivent à $t = 0$; P2 à $t = 20$; P1 à $t = 50$



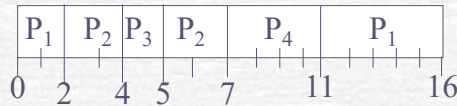
26

26

Shortest-Remaining-Time-First (SRTF)

Exemple

Processus	Tps d'Arrivée	Tps CPU
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4



○ Temps moyen d'attente: P_1 attend de 2 à 11, P_2 de 4 à 5, P_4 de 5 à 7: $(9 + 1 + 0 + 2)/4 = 3$ était 4

○ Temps de rotation moyen: $16 + (7-2) + (5-4) + (11-5) = 7$ était 8

Temps des interruptions

- Temps 0: le seul proc est P_1 , il est choisi
- Temps 2: Interruption causée par l'arrivée de P_2 . Deux procs dans la file: P_1 qui demande encore 5 et P_2 qui demande 4: ce dernier est choisi
- Temps 4: Interruption causée par l'arrivée de P_3 . Trois procs dans la file: $P_1=5$, $P_2=2$, $P_3=1$: ce dernier est choisi, etc compléter comme exercice

Mais, comment déterminer la longueur des cycles à l'avance?

27

27

Shortest-Remaining-Time-First (SRTF)

Exercice 4:

- Donner le diagramme de Gantt, le temps de rotation moyen, et le temps moyen d'attente correspondant au scénario suivant; le cas où les processus arrivent à moment différents:
 - P_1 arrive à temps 0 et dure 6
 - P_2 arrive à temps 1 et dure 2
 - P_3 arrive à temps 2 et dure 8
 - P_4 arrive à temps 3 et dure 4

28

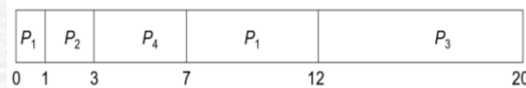
28

Shortest-Remaining-Time-First (SRTF)

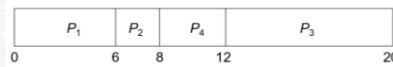
■ Exercice 4:

- Donner le diagramme de Gantt et le temps moyen d'attente correspondant au scenario suivant; le cas où les processus arrivent à moment différents:
 P1 arrive à temps 0 et dure 6
 P2 arrive à temps 1 et dure 2
 P3 arrive à temps 2 et dure 8
 P4 arrive à temps 3 et dure 4

Exercice 4: correction



- Le temps moyen d'attente (SRTF): $((7 - 1) + (1 - 1) + (12 - 2) + (3 - 3))/4 = 4$



- Le temps moyen d'attente (SJF): $(0 + (6 - 1) + (12 - 2) + (8 - 3))/4 = 20/4 = 5$

29

29

Shortest-Remaining-Time-First (SRTF)

➤ Intérêts:

- Minimise le temps d'attente moyen des processus les plus courts

➤ Inconvénients

- Pas de prise en compte de l'importance relative des processus
- Non équité de service : SRT pénalise les processus longs
- Possibilité de famine pour les processus longs

30

30

Algorithmes d'ordonnancement

- Difficultés majeures avec les méthodes discutées
 - Premier arrivé, premier servi, FCFS:
 - Temps moyen d'attente non-optimal
 - Mauvaise utilisation des ressources et famine s'il y a apport continu de processus aux cycles longs (v. effet d'accumulation)
 - SJF et SRTF :
 - Difficulté d'estimer les cycles: faire accompagner la demande d'exécution de chaque programme d'une durée maximum d'exécution autorisée qui est utilisée comme durée d'exécution
 - Famine: risque de famine (les processus de longue durée peuvent n'avoir jamais accès au processeur si des processus de courte durée arrivent en permanence)
 - Donc besoin d'une méthode systématiquement préemptive: Le tourniquet
 - si vous faites toujours les devoirs les plus courts en premier, vous pourriez avoir l'impression de faire beaucoup mais vous pourriez ne jamais arriver aux plus longs...
 - si vous faites les devoirs dans l'ordre d'arrivée, les longs pourraient vous bloquer pour longtemps...
 - donc votre solution est de donner un peu de temps à chacun, cycliquement

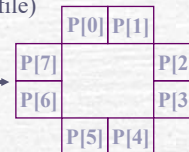
31

31

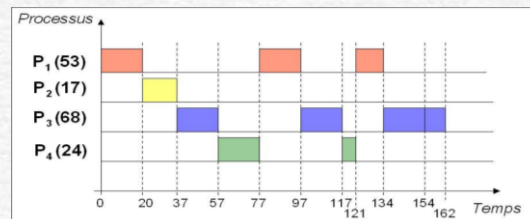
Ordonnancement circulaire (Round-Robin (RR))

- **Principe:** allocation du processeur par tranche (quantum) de temps
 - Chaque processus est alloué une tranche de temps (p.ex. 10-100 millisecons.) pour exécuter
 - S'il exécute pour tranche entière sans autres interruptions, il est interrompu par la minuterie et l'UCT est donnée à un autre processus
 - Le processus interrompu redevient prêt (à la fin de la file)

La file prête est un cercle
(dont RR)



- Exemple ($q=20$, $n=4$)

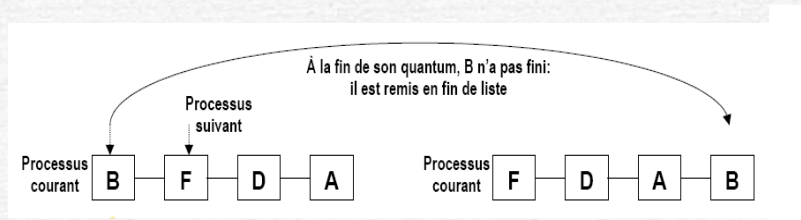


32

32

Ordonnancement circulaire (Round-Robin (RR))

- Chaque processus possède un quantum d'exécution (**plus utilisé en pratique**)
 - Si le processus a fini dans cet intervalle : au suivant !
 - S'il n'a pas fini : le processus passe en fin de liste et au suivant !
- Problème = réglage du quantum :
 - quantum trop petit / commutation (= temps de passage d'un processus à l'autre) : le processeur passe son temps à commuter
 - quantum trop grand : augmentation du temps de réponse d'une commande (même simple)

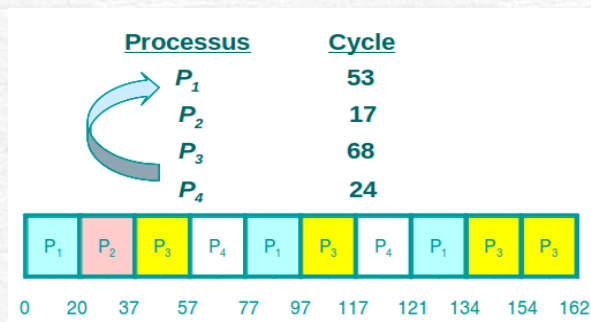


33

33

Ordonnancement circulaire (Round-Robin (RR))

- Exemple: Tourniquet tranche = 20
 - Temps de rotation et temps d'attente moyens beaucoup plus élevés que SJF (exercice: calculez-les...)
 - Mais aucun processus n'est favorisé
 - Typiquement, une moyenne de temps de rotation plus grande que SJF, mais un meilleur *temps de réponse*



34

34

Ordonnancement circulaire (Round-Robin (RR))

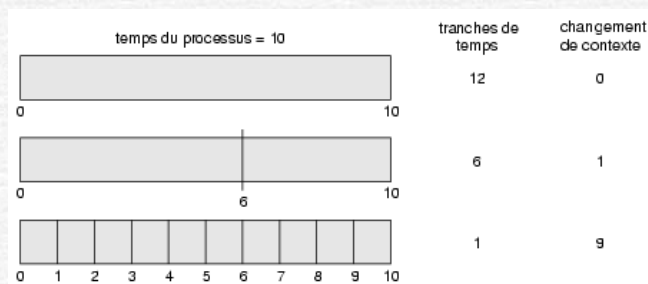
- Chaque processus se voit alloué le CPU pour un temps limité (quantum), en général 10-100 millisecondes. A la fin de ce temps, le processus est arrêté et ajouté à la fin de la file d'attente des processus prêts
- Si n processus sont dans la file d'attente des processus prêts et le quantum est q , alors chaque processus reçoit $1/n$ du temps CPU en parties de q unités. Aucun processus attend plus de $(n-1)q$
- Performance
 - q large $\Rightarrow$ FIFO
 - q petit $\Rightarrow q$ doit être large comparé au temps de commutation de tâche, sinon l'overhead est trop grand

35

35

Ordonnancement circulaire (Round-Robin (RR))

- Une petite tranche augmente les commutations de contexte



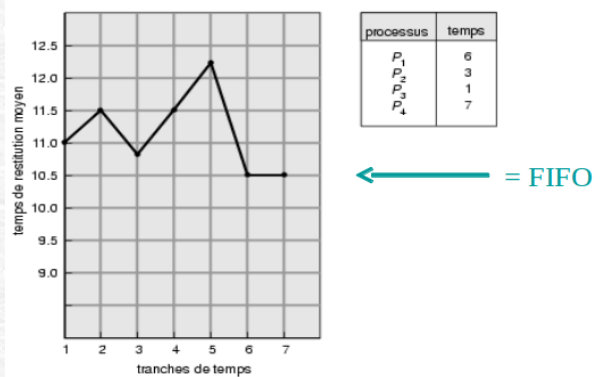
- Exemple pour voir l'importance d'un bon choix de tranche (à développer comme exercice)
 - Trois cycles: A, B, C, toutes de 10
 - Essayer avec: $t=1$ et $t=10$
 - Dans le cas avec $t=10$, tourniquet fonctionne comme FIFO et le temps de rotation moyen est meilleur

36

36

Ordonnancement circulaire (Round-Robin (RR))

- Le temps de rotation (turnaround) varie avec la tranche (exemple à travailler)

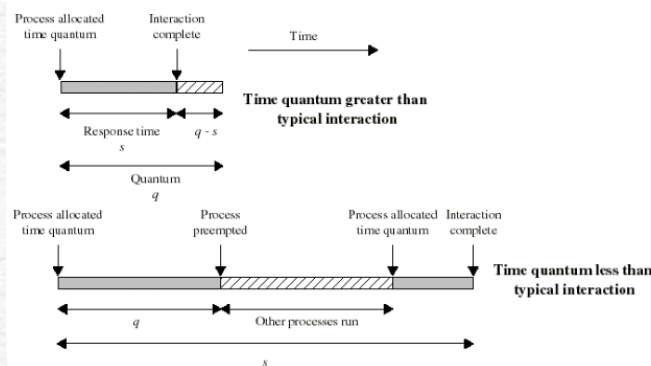


- Exemple qui montre que le temps de rotation moyen n'améliore pas nécessairement en augmentant la tranche (sans considérer les temps de commutation contexte)

37

Ordonnancement circulaire (Round-Robin (RR))

- Choix de la tranche pour le tourniquet
 - doit être beaucoup plus grande que le temps requis pour exécuter le changement de contexte
 - devrait permettre le complètement d'approx. 80% des cycles pour donner le temps à la plupart des proc de terminer leur cycle, mais pas trop pour éviter de pénaliser les processus courts. Voir ex. préc. où les tranches optimales sont 6 et 7



38

Ordonnancement circulaire (Round-Robin (RR))

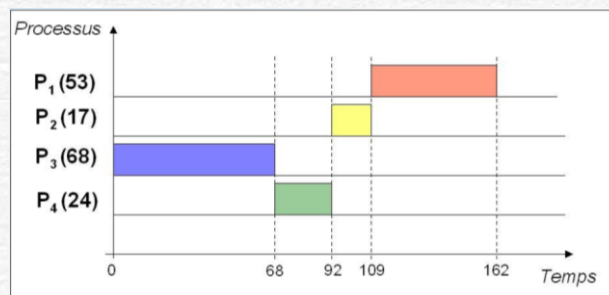
- Comment déterminer la tranche en pratique
 - Évidemment la durée médiane des cycles des processus en attente est impossible à déterminer!
 - En pratique, les observations passées peuvent être utilisées: le SE peut garder une trace des durées des cycles des processus récemment exécutés et ajuster la tranche périodiquement
- Priorité
 - Affectation d'une priorité à chaque processus (p.ex. un nombre entier)
 - souvent les petits chiffres dénotent des hautes priorités: 0 la plus haute
 - L'UCT est donnée au processus prêt avec la plus haute priorité
- Problème possible avec les priorités
 - Famine: les processus moins prioritaires n'arrivent jamais à exécuter
- Solution: vieillissement:
 - modifier la priorité d'un processus en fonction de son âge et de son historique d'exécution
 - le processus change de file d'attente
- **Plus en général, la modification dynamique des priorités est une politique souvent utilisée**

39

39

Ordonnancement à priorités statiques

- **Principe** : allocation du processeur selon des priorités statiques (numéros affectés aux processus pour toute la vie de l'application)
- **Exemple**: priorités(P_1, P_2, P_3, P_4)=(3,2,0,1)



40

40

Ordonnement à priorités statiques

Exercice 4:

- Donner le diagramme de Gantt, le temps de rotation moyen, et le temps moyen d'attente correspondant au scénario suivant; le cas où les processus arrivent à temps 0:

P1 dure 9, priorité 3

P2 dure 2, priorité 4

P3 dure 1, priorité 1

P4 dure 5, priorité 3

P5 dure 6, priorité 2

41

41

Ordonnement à priorités statiques

Exercice 4:

- Donner le diagramme de Gantt et le temps moyen d'attente correspondant au scénario suivant; le cas où les processus arrivent à temps 0:

P1 dure 9, priorité 3

P2 dure 2, priorité 4

P3 dure 1, priorité 1

P4 dure 5, priorité 3

P5 dure 6, priorité 2

Exercice 4: correction

P_3	P_5	P_1	P_4	P_2	
0	1	7	16	21	23

- Temps moyen d'attente: $(7 + 21 + 0 + 16 + 1)/5 = 9$

42

42

Ordonnement à priorités statiques

- **Intérêts:**
 - Prise en compte de l'importance relative des processus
 - Mise en œuvre simple
- **Inconvénients:**
 - Problèmes de blocages durant des périodes de temps illimitées
 - Problèmes de famines des processus de moindres priorités

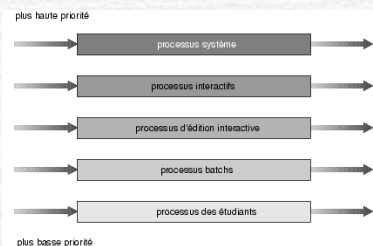
43

43

Ordonnement avec files multiples

- La file prêt est séparée en plusieurs files, p.ex.
 - travaux `d'arrière-plan` (background - batch)
 - travaux `de premier plan` (foreground - interactive)
- Chaque file a son propre algorithme d'ordonnement, p.ex.
 - FCFS pour arrière-plan
 - tourniquet pour premier plan
- Comment ordonnancer entre files?
 - Priorité fixe à chaque file : famine possible, ou
 - Chaque file reçoit un certain pourcentage de temps UCT, p.ex.
 - 80% pour arrière-plan
 - 20% pour premier plan

Un proc peut être servi seulement si toutes les files de priorités plus hautes sont vides

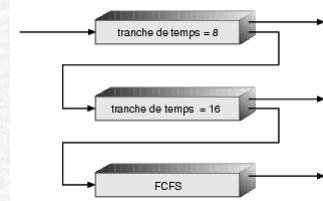


44

44

Ordonnancement avec files multiples et à retour

- Un processus peut passer d'une file à l'autre, p.ex. quand il a passé trop de temps dans une file
- À déterminer:
 - nombre de files, algorithmes d'ordonnancement pour chaque file
 - algorithmes pour décider quand un proc doit passer d'une file à l'autre
 - algorithme pour déterminer, pour un proc qui devient prêt, sur quelle file il doit être mis
- Exemple de files multiples à retour
 - Trois files:
 - P0: tourniquet, tranche = 8 msec
 - P1: tourniquet, tranche = 16 msec
 - P2: FCFS
 - Ordonnancement:
 - Un nouveau processus entre dans Q0, il reçoit 8 msec d'UCT
 - S'il ne finit pas dans les 8 msec, il est mis dans Q1, il reçoit 16 msec additionnels
 - S'il ne finit pas encore, il est interrompu et mis dans Q2
 - Si plus tard il commence à avoir des cycles plus courts, il pourrait retourner à Q0 ou Q1



45

45

Ordonnancement avec files multiples et à retour

- Exercice 5
 - Dans un système avec files multiples, un processus a besoin de 50 secondes d'UCT (sans appels au SE).
 - Dans la première file on lui donne 5 secondes et à chaque file successive on lui donne le double (10, 20, 40 ...)
 - Combien de fois sera-t-il interrompu et dans quelle file se trouvera-t-il quand il terminera?

46

46

Ordonnancement avec files multiples et à retour

- En pratique...
 - Les méthodes que nous avons vu sont toutes utilisées (sauf plus court servi pur qui est normalement impossible, pourquoi?)
 - Les SE sophistiqués fournissent aux gérants de grands systèmes des librairies de méthodes, qu'ils peuvent choisir et combiner au besoin après avoir observé le comportement du système
 - Pour chaque méthode, plusieurs params sont disponibles: p.ex. durée des tranches, coefficients, etc.
 - Ces méthodes évidemment sont importantes surtout pour les grands ordis qui ont des fortes charges de travail

47

47

Ordonnancement avec files multiples et à retour

- De plus
 - Nous n'avons pas considéré en détail tous les problèmes qui se présentent dans l'ordonnancement UCT. P.ex. les ordonnanceurs UCT ne peuvent pas donner l'UCT à un processus pour tout le temps dont il a besoin Car en pratique, l'UCT sera souvent interrompue par quelque événement externe avant la fin de son cycle
 - Cependant les mêmes principes d'ordonnancement s'appliquent à unités qui ne peuvent pas être interrompues, comme une imprimante, une unité disque, etc. Dans le cas de ces unités, on pourrait avoir des infos complètes concernant le temps de cycle prévu, etc. Aussi, cette étude ne considère pas du tout les temps d'exécution de l'ordonnanceur, etc.

48

48

Tableau de comparaison

	Critère sélection	Préempt	Motivation	Temps de rotat. et atten.	Temps de système	Effect sur processus	Famine
PAPS FCFS	Max [w]	non	Simplicité	Variable	Minim.	Favor. proc. trib. UCT	Non
Tourniq.	Tour fixe	oui	Equité	Variable selon tranche, Normalement élevé	Élevé si tranches courtes	Équitable	Non
PCS SJF	Min[s]	non	Optimisation des temps	Optimal, mais souffre si des proc. longs arrivent au début	Peut être élevé (pour estimer les longs. des trames)	Bon pour proc. courts, pénalise proc. longs	Possible
PCS SJF préemp.	Min[s-e]	oui	Évite le problème de proc. longs au début	Meilleur, même si des proc. longs arrivent au début	Peut être élevé	Pénalise plus encore proc. longs	Possible
Files multiniv	v. détails	oui	Varie la longueur des tranches en fonction des besoins	Variable	Peut être élevé	Variable	Peut être évitée

w: temps total dans système jusqu'à présent;

e: temps en exec jusqu'à présent

s: temps demandé;

Famine est 'renvoi infini'

49

49

References

- Luigi Logrippo, Ordonnancement Processus, slides. <http://w3.uqo.ca/luigi/>
- <http://perso.univ-rennes1.fr/pascal.gentil/docs/unix/unix.fichiers.html#cat1>
- <http://upsilon.cc/~zack/teaching/1314/progsyst/#index1h2>
- Karim Sehaba, Systèmes et Architectures, <http://perso.univ-lr.fr/ksehaba/>
- Vincent Granet, Introduction à Linux, Polytech'Nice-Sophia
- Sylvain CHERRIER, Cours Linux,
- Gestion des processus, Telecom-ParisTech BCI Informatique

50

50